SQL QUERY IMPLEMENTATION FOR E-COMMERCE DATABASE

SUBQUERIES AND NESTED QUERIES

A subquery is a query placed inside another query. The inner query calculates a value, and the outer query uses that value to filter results.

1. Orders Above Average Order Value

```
SQL> SELECT Order_ID, Customer_ID, Total_Amount
   2  FROM Orders
   3  WHERE Total_Amount > (SELECT AVG(Total_Amount) FROM Orders)
   4  ORDER BY Total_Amount DESC;

ORDER_ID  CUSTOMER_ID  TOTAL_AMOUNT
--------  -----------  ------------
    5008            8       7997.00
    5004            4       4997.00
    5007            7       4498.00
    5002            2       3998.00
    5006            6       3998.00
    5011            4       3698.00
    5001            1       3697.00

7 rows selected.
```

2. Products Priced Above Average

```
SQL> SELECT Product_ID, Product_Name, Price
   2  FROM Product
   3  WHERE Price > (SELECT AVG(Price) FROM Product)
   4  ORDER BY Price DESC;

PRODUCT_ID  PRODUCT_NAME       PRICE
----------  ---------------  -------
       105  Running Shoes    3999.00
       114  Denim Jacket     3299.00
       106  Sports Sneakers  2999.00
       108  Analog Watch     2799.00
       101  Slim Fit Jeans   2499.00
       110  Cotton Saree     2199.00

6 rows selected.
```

3. Products Priced Below Average

```
SQL> SELECT Product_ID, Product_Name, Price
   2  FROM Product
   3  WHERE Price < (SELECT AVG(Price) FROM Product)
   4  ORDER BY Price ASC;

PRODUCT_ID  PRODUCT_NAME          PRICE
----------  -------------------  -------
       113  Flip Flops            499.00
       109  Graphic T-Shirt       599.00
       107  Leather Wallet        699.00
       102  Cotton Casual Shirt   899.00
       112  Canvas Backpack      1199.00
       104  Floral Summer Dress  1299.00
       111  Track Pants          1499.00
       103  Printed Kurta Set    1799.00

8 rows selected.
```

4. Step 1: Total Purchase per Customer

```
SQL> SELECT Customer_ID, SUM(Total_Amount) AS Total_Purchase
   2  FROM Orders
   3  GROUP BY Customer_ID
   4  ORDER BY Customer_ID;

CUSTOMER_ID  TOTAL_PURCHASE
-----------  --------------
          1         6695.00
          2         3998.00
          3         1598.00
          4         8695.00
          5         2397.00
          6         3998.00
          7         4498.00
          8         7997.00
          9         2397.00
         10         1698.00

10 rows selected.
```

5. Step 2: Maximum Total Purchase

```
SQL> SELECT MAX(Total_Purchase) AS Max_Purchase
   2  FROM (SELECT Customer_ID, SUM(Total_Amount) AS Total_Purchase
   3        FROM Orders
   4        GROUP BY Customer_ID) t;

MAX_PURCHASE
------------
     8695.00

1 rows selected.
```

6. Step 3: Customer with Maximum Purchase

```
SQL> SELECT c.Customer_ID, c.First_Name, SUM(o.Total_Amount) AS Total_Purchase
   2  FROM Customer c
   3  JOIN Orders o ON c.Customer_ID = o.Customer_ID
   4  GROUP BY c.Customer_ID, c.First_Name
   5  HAVING SUM(o.Total_Amount) = (SELECT MAX(Total_Purchase)
   6                                FROM (SELECT Customer_ID, SUM(Total_Amount) AS Total_Purchase
   7                                      FROM Orders
   8                                      GROUP BY Customer_ID) t);

CUSTOMER_ID  FIRST_NAME  TOTAL_PURCHASE
-----------  ----------  --------------
          4  Karthik            8695.00

1 rows selected.
```

COMPLEX BUSINESS QUERIES

7. Customers with Order Value Above Average Order Value

```
SQL> SELECT c.Customer_ID, c.First_Name, o.Order_ID, o.Total_Amount
   2  FROM Customer c
   3  JOIN Orders o ON c.Customer_ID = o.Customer_ID
   4  WHERE o.Total_Amount > (SELECT AVG(Total_Amount) FROM Orders)
   5  ORDER BY o.Total_Amount DESC;

CUSTOMER_ID  FIRST_NAME  ORDER_ID  TOTAL_AMOUNT
-----------  ----------  --------  ------------
          8  Meena           5008       7997.00
          4  Karthik         5004       4997.00
          7  Arun            5007       4498.00
          2  Hema            5002       3998.00
          6  Divya           5006       3998.00
          4  Karthik         5011       3698.00
          1  Nivethitha      5001       3697.00

7 rows selected.
```

8. Products Whose Sales Are Above Average

Sales of a product are calculated from the Order_Item table as the sum of Quantity x Unit_Price.
The subquery finds the average sales across all products.

```
SQL> SELECT p.Product_ID, p.Product_Name, SUM(oi.Quantity * oi.Unit_Price) AS Total_Sales
   2  FROM Product p
   3  JOIN Order_Item oi ON p.Product_ID = oi.Product_ID
   4  GROUP BY p.Product_ID, p.Product_Name
   5  HAVING SUM(oi.Quantity * oi.Unit_Price) >
   6         (SELECT AVG(Product_Sales)
   7          FROM (SELECT Product_ID, SUM(Quantity * Unit_Price) AS Product_Sales
   8                FROM Order_Item
   9                GROUP BY Product_ID) t)
  10  ORDER BY Total_Sales DESC;

PRODUCT_ID  PRODUCT_NAME       TOTAL_SALES
----------  -----------------  -----------
       105  Running Shoes          7998.00
       110  Cotton Saree           4398.00
       103  Printed Kurta Set      3598.00
       112  Canvas Backpack        3597.00
       114  Denim Jacket           3299.00

5 rows selected.
```

ADVANCED SALES REPORTS

9. Customer Purchase Report

```
SQL> SELECT c.Customer_ID, c.First_Name,
   2         COUNT(o.Order_ID) AS Orders,
   3         SUM(o.Total_Amount) AS Total_Purchase,
   4         ROUND(AVG(o.Total_Amount), 2) AS Avg_Order_Value
   5  FROM Customer c
   6  JOIN Orders o ON c.Customer_ID = o.Customer_ID
   7  GROUP BY c.Customer_ID, c.First_Name
   8  ORDER BY c.Customer_ID;

CUSTOMER_ID  FIRST_NAME  ORDERS  TOTAL_PURCHASE  AVG_ORDER_VALUE
-----------  ----------  ------  --------------  ---------------
          1  Nivethitha       2         6695.00          3347.50
          2  Hema             1         3998.00          3998.00
          3  Ravi             1         1598.00          1598.00
          4  Karthik          2         8695.00          4347.50
          5  Priya            1         2397.00          2397.00
          6  Divya            1         3998.00          3998.00
          7  Arun             1         4498.00          4498.00
          8  Meena            1         7997.00          7997.00
          9  Nisha            1         2397.00          2397.00
         10  Suresh           1         1698.00          1698.00

10 rows selected.
```

10. Advanced Product Sales Report

```
SQL> SELECT p.Product_ID, p.Product_Name,
   2         COUNT(DISTINCT oi.Order_ID) AS Orders,
   3         SUM(oi.Quantity) AS Qty_Sold,
   4         SUM(oi.Quantity * oi.Unit_Price) AS Total_Sales,
   5         ROUND(AVG(oi.Unit_Price), 2) AS Avg_Price
   6  FROM Product p
   7  JOIN Order_Item oi ON p.Product_ID = oi.Product_ID
   8  GROUP BY p.Product_ID, p.Product_Name
   9  ORDER BY Total_Sales DESC;

PRODUCT_ID  PRODUCT_NAME         ORDERS  QTY_SOLD  TOTAL_SALES  AVG_PRICE
----------  -------------------  ------  --------  -----------  ---------
       105  Running Shoes             2         2      7998.00    3999.00
       110  Cotton Saree              2         2      4398.00    2199.00
       103  Printed Kurta Set         2         2      3598.00    1799.00
       112  Canvas Backpack           3         3      3597.00    1199.00
       114  Denim Jacket              1         1      3299.00    3299.00
       106  Sports Sneakers           1         1      2999.00    2999.00
       111  Track Pants               2         2      2998.00    1499.00
       108  Analog Watch              1         1      2799.00    2799.00
       102  Cotton Casual Shirt       2         3      2697.00     899.00
       101  Slim Fit Jeans            1         1      2499.00    2499.00
       109  Graphic T-Shirt           3         4      2396.00     599.00
       113  Flip Flops                3         4      1996.00     499.00
       107  Leather Wallet            2         2      1398.00     699.00
       104  Floral Summer Dress       1         1      1299.00    1299.00

14 rows selected.
```

11. Category-wise Advanced Sales Report

```
SQL> SELECT cat.Category_ID, cat.Category_Name,
   2         COUNT(DISTINCT oi.Order_ID) AS Orders,
   3         SUM(oi.Quantity) AS Qty_Sold,
   4         SUM(oi.Quantity * oi.Unit_Price) AS Total_Sales,
   5         ROUND(AVG(oi.Unit_Price), 2) AS Avg_Price
   6  FROM Category cat
   7  JOIN Product p ON cat.Category_ID = p.Category_ID
   8  JOIN Order_Item oi ON p.Product_ID = oi.Product_ID
   9  GROUP BY cat.Category_ID, cat.Category_Name
  10  ORDER BY Total_Sales DESC;

CATEGORY_ID  CATEGORY_NAME  ORDERS  QTY_SOLD  TOTAL_SALES  AVG_PRICE
-----------  -------------  ------  --------  -----------  ---------
          1  Menswear            7        11     13889.00    1376.78
          3  Footwear            5         7     12993.00    2082.33
          2  Womenswear          4         5      9295.00    1859.00
          4  Accessories         5         6      7794.00    1299.00

4 rows selected.
```